

UNIDAD 1

LENGUAJE ESTRUCTURADO

Evolución de los lenguajes de programación

Año	Software	Año	Software
1843	Primer lenguaje de Programación	1993	Ruby, Lua
1940	Lenguaje máquina	1994	CLOS
1950	Lenguaje ensamblador	1995	Java, JavaScript, PHP, Delphi (Object Pascal)
1957	Fortran	1996	WebDNA
1958	LIPS	1997	Rebol
1959	Cobol	1999	D
1964	Basic	2000	ActionScript
1970	Pascal	2001	C#, Visual Basic.NET
1972	C	2002	F#
1980	Ada	2003	Groovy, Scala, Factor
1983	C++	2005	Scratch
1984	Matlab	2007	Clojure
1987	Perl	2009	Go
1988	Mathematica	2010	Kotlin
1990	Haskell	2011	Dart
1991	Python, Visual Basic, HTML	2014	Swift

LENGUAJE NO ESTRUCTURADO (BASIC)

- No se conserva un orden de manera clara
- No son claros los bloques de instrucciones
- Cada línea tiene un número.

```
LIST
10 U=1024 : REM VIDEO MEMORY
15 W=40 : REM SCREEN WIDTH
20 DIM X(99),Y(99) : N=0
30 FOR Y=0 TO 24 : FOR X=1 TO W-1
40 P=U+Y*W+X
50 C1=PEEK(P):C2=PEEK(P+1):C0=PEEK(P-1)
60 IF C0<>32 OR C1<48 OR C2>57 THEN 100
70 I=C1-48
80 IF C2<>32 THEN I=I*10+C2-48
90 X(I)=X:Y(I)=Y:IF I>N THEN N=I
100 NEXT X : NEXT Y
110 CS="\ / | + | - \"
120 X=X(1):Y=Y(1)
130 FOR I=1 TO N
140 DX=SGN(X(I)-X)
150 DY=SGN(Y(I)-Y)
160 A=ASC(MID$(CS,5+DX*3+DY))-128
170 X=X+DX:Y=Y+DY
180 POKE U+Y*W+X,A
190 IF DX<>0 OR DY<>0 THEN 140
200 NEXT I
READY.
```


LENGUAJE NO ESTRUCTURADO (Fortran)

- No se conserva un orden de manera clara
- No son claros los bloques de instrucciones
- Cada línea tiene un número.

```
1      PROGRAM PRINCIPAL
2          PARAMETER (TAMMÁX=99)
3          REAL A(TAMMAX)
4      10      READ (5,100,END=999) K
5      100     FORMAT(I5)
6              IF (K.LE.0.OR K.GT.TAMMÁX) STOP
7              READ *,(A(I),I=1,K)
8              PRINT *,(A(I),I=1,K)
9              PRINT * , 'SUMA=', SUM(A,K)
10             GO TO 10
11      99      PRINT * , "Todo listo"
12             STOP
13             END
14  SUBPROGRAMA DE SUMATORIA EN C
15      FUNCTION SUM(V,N)
16          REAL :: V(N) ! Declaración de estilo nuevo
17          SUM = 0.0
18          DO 20 I = 1,N
19              SUM = SUM + V(I)
20      20      CONTINUE
21              RETURN
22              END
```


LENGUAJE NO ESTRUCTURADO (Ensamblador)

- No se conserva un orden de manera clara
- No son claros los bloques de instrucciones
- Cada línea tiene un número.

```
00001A1E 4D 4B      LDR
00001A20 E3 58      LDR
00001A22 1B 68      LDR
00001A24 18 46      MOV
00001A26 FF F7 52 EA BLX
00001A2A 03 46      MOV
00001A2C 18 46      MOV
00001A2E 4A 4B      LDR
00001A30 7B 44      ADD
00001A32 19 46      MOV
00001A34 00 F0 34 FB BL
00001A38 48 4B      LDR
00001A3A E3 58      LDR
00001A3C 1B 68      LDR
00001A3E 18 46      MOV
00001A40 FF F7 44 EA BLX
00001A44 02 46      MOV
00001A46 07 F5 43 63 ADD.W
00001A4A 10 46      MOV
00001A4C 19 46      MOV
00001A4E 4F F0 02 02 MOV.W
00001A52 4F F0 0A 03 MOV.W
00001A56 00 F0 77 FB BL
00001A5A 03 46      MOV
00001A5C 00 2B      CMP

R3, =(stdout_ptr - 0xC000)
R3, [R4,R3] ; stdout
R3, [R3]
R0, R3 ; stream
fileno
R3, R0
R0, R3
R3, =(a1ChangeDisplay - 0x1
R3, PC ; "1 ) Change displ
R1, R3
print
R3, =(stdin_ptr - 0xC000)
R3, [R4,R3] ; stdin
R3, [R3]
R0, R3 ; stream
fileno
R2, R0
R3, R7, #0xC30
R0, R2
R1, R3
R2, #2
R3, #0xA
read
R3, R0
R3, #0
```


Lenguaje Estructurado C/C++

- C++ Es un lenguaje estructurado
- Derivado de ANSI C
- Enfocado a programación utilizando Objetos
- Lenguaje utilizado como base en la generación de otros lenguajes.

```
3.FirstSteps > 3.4ErrorsWarnings > main.cpp > main()
1  #include <iostream>
2
3  int main()
4  {
5      int numero = 0;
6      std::cout << "Division de numero entre cero en una variable" << std::endl;
7      std::cout << 100 / numero << std::endl;
8      std::cout << "Final de la división entre cero" << std::endl; }?
9      return 0;
10 }
```

TERMINAL SALIDA CONSOLA DE DEPURACIÓN PROBLEMAS

Division de numero entre cero en una variable
PS C:\C20-Course\3.FirstSteps\3.4ErrorsWarnings>

Lenguaje Estructurado Java

```
1 //Codificado por sAf0rAs
2 public class SyGContarElementosRepetidos{
3     static int A=10;
4     static int B=20;
5     static int vectorA[]=new int[A];
6     static int vectorB[]=new int[B];
7     static int elemA=0;
8     static int elemB=0;
9     static int z=0;
10    static void llenaArreglo(){
11        for(int i=0;i<vectorA.length;i++){
12            vectorA[i]=(int)(Math.random()*100+1);
13        }
14
15        for(int i=0;i<vectorB.length;i++){
16            vectorB[i]=(int)(Math.random()*100+1);
17        }
18    }
19
20
21    static void compara(){
22        for(int i=0;i<vectorA.length;i++){
23            for(int j=0;j<vectorB.length;j++){
24                if(vectorA[i]==vectorB[j])
25                    elemA++;
26            }
27            System.out.println("El elemento "+vectorA[i]+
28                elemA=0;
29        }
30    }
31
32    static void imprimir(){
33        for(int i=0;i<vectorA.length;i++){
34            System.out.print("A"+"["+i+"]=" +vectorA[i]+
35        }
36        for(int i=0;i<vectorB.length;i++){
37            System.out.print("B"+"["+i+"]="+vectorB[i]+"\\
38        }
39    }
40
41    public static void main(String[] args){
42        llenaArreglo();
43        imprimir();
44        compara();
45    }
46
47 }
```


Lenguaje Estructurado Pascal

```
PROGRAM EJER5;  
  USES CRT;  
  VAR num, flag, x: INTEGER;  
      resp: CHAR;  
BEGIN  
  ClrScr;  
  num:=0;  
  x:=1;  
  FOR num:=1 TO 100 DO  
  BEGIN  
    IF (num mod 20)= 0 THEN  
      flag := x;  
      WRITELN (num);  
      IF flag = x THEN  
      BEGIN  
        WRITE('DESEA CONTINUAR: S/N --> '); READLN(resp);  
        IF UPCASE (resp)<>'S' THEN  
        BEGIN  
          WRITE ('Este programa ha finalizado'); EXIT  
        END;  
      END;  
      x:= x + 20;  
    END;  
  END;  
  READLN  
END.
```


Operadores

- Un operador es un elemento de programa que se aplica a uno o varios operandos en una expresión o instrucción.
- Los operadores que requieren un operando (incremento, decremento) se conocen como operadores unarios.
- Los operadores que requieren dos operandos (+, -, *, /) se conocen como operadores binarios.
- Un operador, el operador condicional (?:), utiliza tres operandos y es el único operador ternario de C++”
- <http://msdn.microsoft.com/eses/library/ms173145.aspx>

OPERADORES DE ASIGNACIÓN

- `=` Asignación Básica `x=6`
- `*=` Asigna Producto `x*=5`
- `/=` Asigna División `x/=5`
- `+=` Asigna Suma `x+=5`
- `-=` Asigna Resta `x-=5`
- `%=` Asigna Modulo `x%=5`
- `<<=` Asigna Desplazamiento Izquierda `x<<=5`
- `>>=` Asigna Desplazamiento Derecha `x>>=5`
- `&=` Asigna AND entre Bits `x&=2`
- `^=` Asigna XOR entre Bits `x^= 1`
- `|=` Asigna OR entre Bits `x|=3`

OPERADORES ARITMÉTICOS

- – Resta
- + Suma
- * Multiplicación
- / División
- % Módulo
- -- Decremento
- ++ Incremento

Ejemplo 1

```
int x,y;
```

```
x = 2004;
```

```
y = ++x;
```

Cuánto vale x y y?

Ejemplo 2

```
int x,y;
```

```
x = 2004;
```

```
y = x++;
```

Cuánto valen x y y?

Ejemplo 2

```
int x,y;  
x = 2004;  
y = x++;
```

Cuánto valen x y y?

```
int x,y;  
x = 2004;  
y = x;  
x=x+1; //x++
```


Ejemplo 2b

```
int x,y;
```

```
x = 2004;
```

```
y = ++x;
```

Cuánto valen x y y?

Ejemplo 2b

```
int x,y;  
x = 2004;  
y = ++x;
```

Cuánto valen x y y?

```
int x,y;  
x = 2004;  
x=x+1; //x++  
y = x;
```


Operadores para punteros

- - Desplazamiento descendente
- + Desplazamiento ascendente
- - Distancia entre elementos
- -- Desplazamiento descendente de 1 elemento
- ++ Desplazamiento ascendente de 1 elemento

Operadores Relacionales

- $<$ Menor que
- $>$ Mayor que
- $\leq$ Menor o igual
- $\geq$ Mayor o igual
- $==$ Igual
- $\neq$ Diferente

Operadores Relacionales

- && And lógico
- || Or lógico
- ! Negación lógica

Condicionales

- En programación, los condicionales son procedimientos que se usan para decidir si se ejecuta o no un bloque de instrucciones.
- Las condiciones se definen utilizando los operadores relacionales.

Condicionales

- La forma más común de usar el condicional es con la instrucción *if*. Una descripción de su uso considerando únicamente cuando la condición es cierta (true):

```
if(condición o prueba lógica)
```

```
    Lo que pasa si se cumple la condición
```


Condicionales

- También puede llevar sentencias o acciones cuando no se cumpla (condición = false).

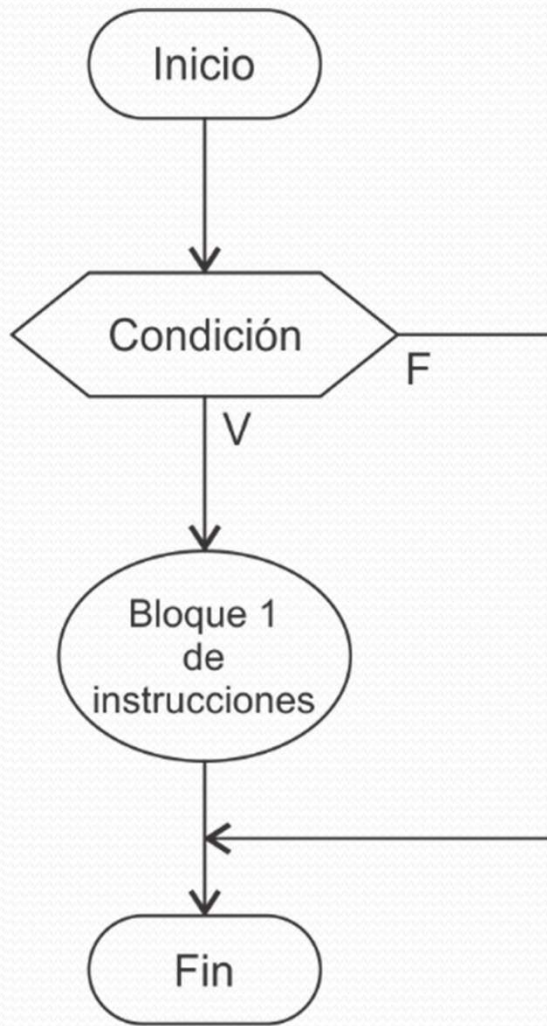
```
if(condición)
```

```
    Lo que pasa si se cumple la condición
```

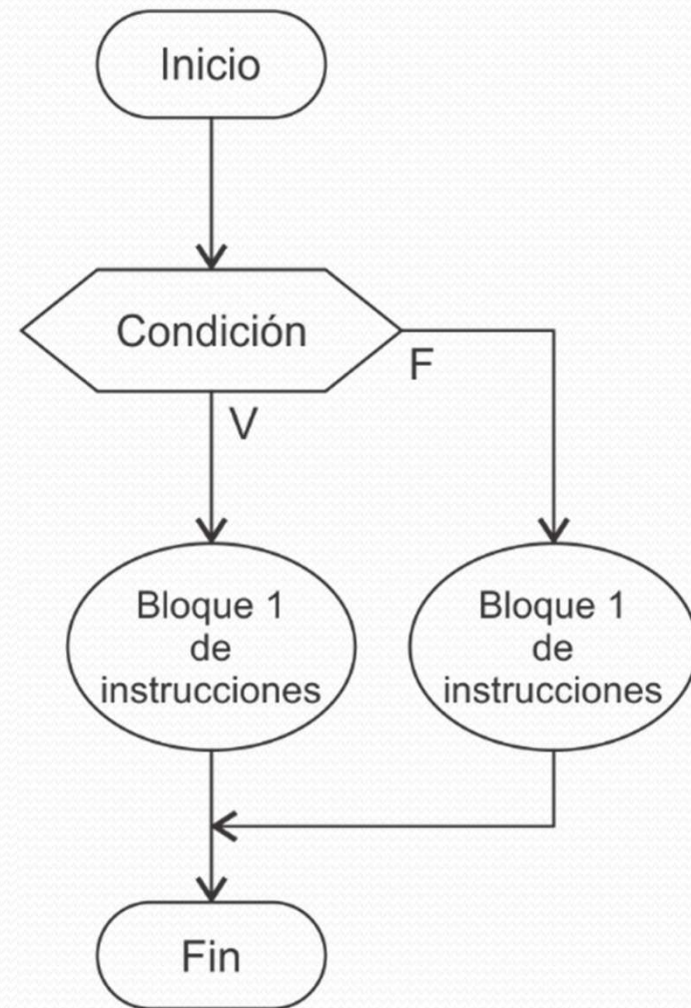
```
else
```

```
    Lo que pasa si no se cumple la  
condición
```


Condicionales



(a)



(b)

Ejemplo de toma de decisiones

- Algoritmo 1.1.
- Uso del if con solo respuesta verdadera.

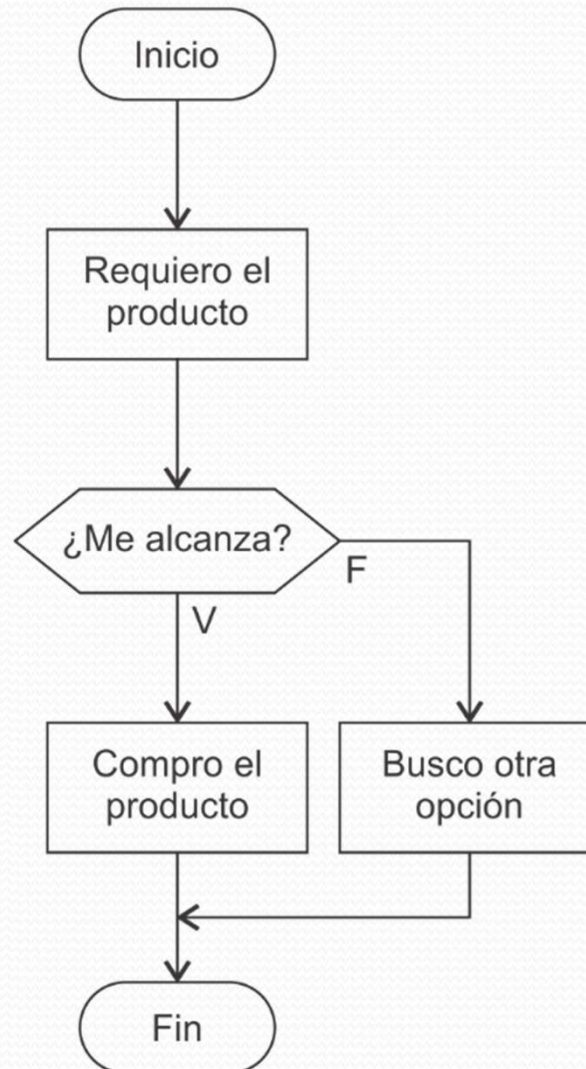
1. `if (Dinero disponible >= costo)`
2. Se adquiere el producto.
3. Se retira del establecimiento.

Ejemplo de toma de decisiones

- Algoritmo 1.2.
- Funcionamiento del if considerando el resultado de la condición cierto y falso.

```
1. if (Dinero disponible >= costo)
2.     Se adquiere el producto.
3. Else
4.     Se busca otra opción de compra.
5. Se retira de la tienda.
```


Ejemplo de toma de decisiones



Condiciones

- La Instrucción IF sirve para alterar la secuencia del programa, dependiendo del resultado de una condición, utilizando los operadores relacionales.

```
if(condición){  
    //Bloque de operaciones 1  
}  
else{  
    //Bloque de operaciones 2  
}
```

1.1.2 Ciclos y Condiciones.

- Ciclo FOR

```
For(valores iniciales; condición; incremento){  
    //Bloque de instrucciones  
}
```

Ejemplo:

```
X=0;  
for(i=0;i<10;i++){  
    x+=i;  
}
```


1.1.2 Ciclos y Condiciones.

- Ciclo DO

```
do{  
    //Bloque de instrucciones  
}  
While (condición);
```

Ejemplo:

```
X=0; i=0;  
do{  
    x+=i;  
    i++;  
}  
while(i<10);
```

1.1.2 Ciclos y Condiciones.

- Ciclo WHILE

```
While(condición){  
    //Bloque de instrucciones  
}
```

Ejemplo:

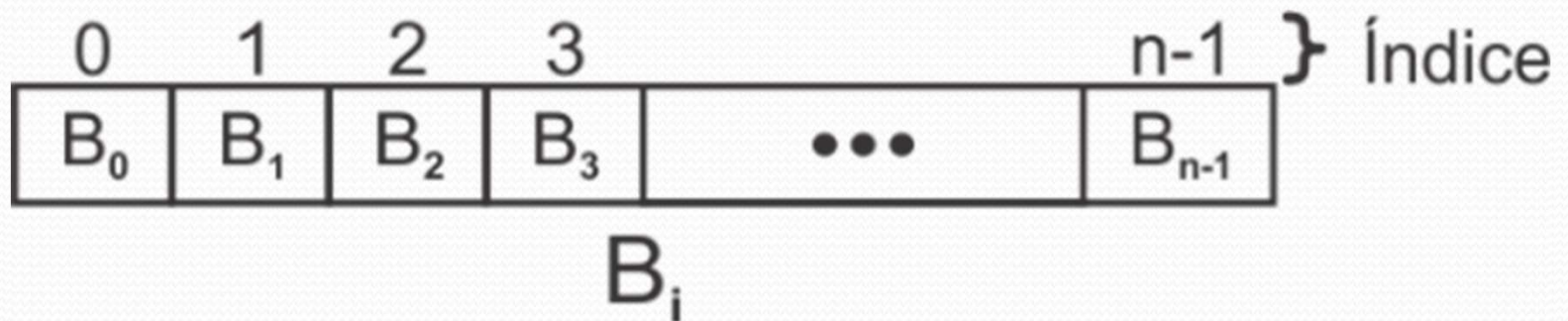
```
X=0; i=1;  
While(i<10){  
    x+=i;  
    i++;  
};
```


1.1.3 Arreglos

- Se conocen también como Arrays, vectores o matrices.
- Se utilizan para almacenar múltiples datos de un mismo tipo en una misma variable.
- Lenguaje C++ un arreglo se le conoce como un tipo de dato compuesto.
- Los arreglos pueden tener una o varias dimensiones.

Arreglo de 1 dimensión (Vector)

- Se requiere de solamente 1 índice para posicionarse en la localidad de memoria del registro a trabajar.



Arreglo de 2 dimensiones (Matriz)

- Son arreglos que requieren de dos índices para posicionarse en la dirección de memoria a trabajar.

Índice de columna

	0	1	2	3	...	n-1
0	$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$	...	$A_{0,n-1}$
1	$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	...	$A_{1,n-1}$
2	$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$	...	$A_{2,n-1}$
3	$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$	...	$A_{3,n-1}$
...	...	...	...	...	...	...
m-1	$A_{m-1,0}$	$A_{m-1,1}$	$A_{m-1,2}$	$A_{m-1,3}$	...	$A_{m-1,n-1}$

Índice de renglón

$A_{i,j}$

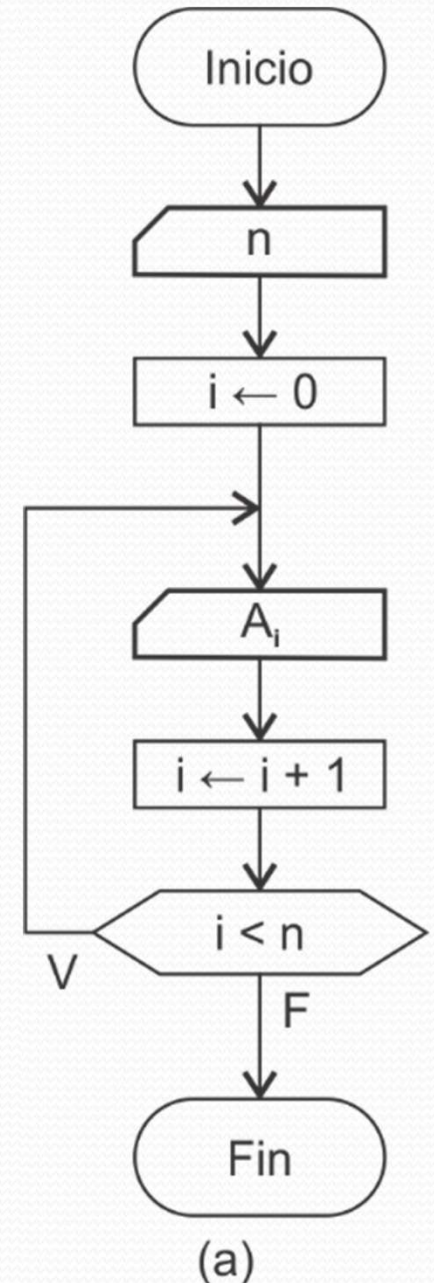
Arreglos unidimensionales.

- El Algoritmo 1.8 muestra la manera de asignar datos a un arreglo unidimensional (vector).

1. Se define usar la variable A como el arreglo
2. Se pide el número de datos y se almacena en n
3. Inicia el índice i en 0
4. Se pide el valor de A_i
5. Se incrementa el valor de i en 1
6. ¿Es $i < n$?
Si: ir al paso 4
No: ir al paso 7
7. Fin

Arreglos unidimensionales.

- Asignación de datos a un vector.
(a) Usando do - while. (b) Usando while. (c) Usando for.



Código C para asignar datos a un arreglo con do - while

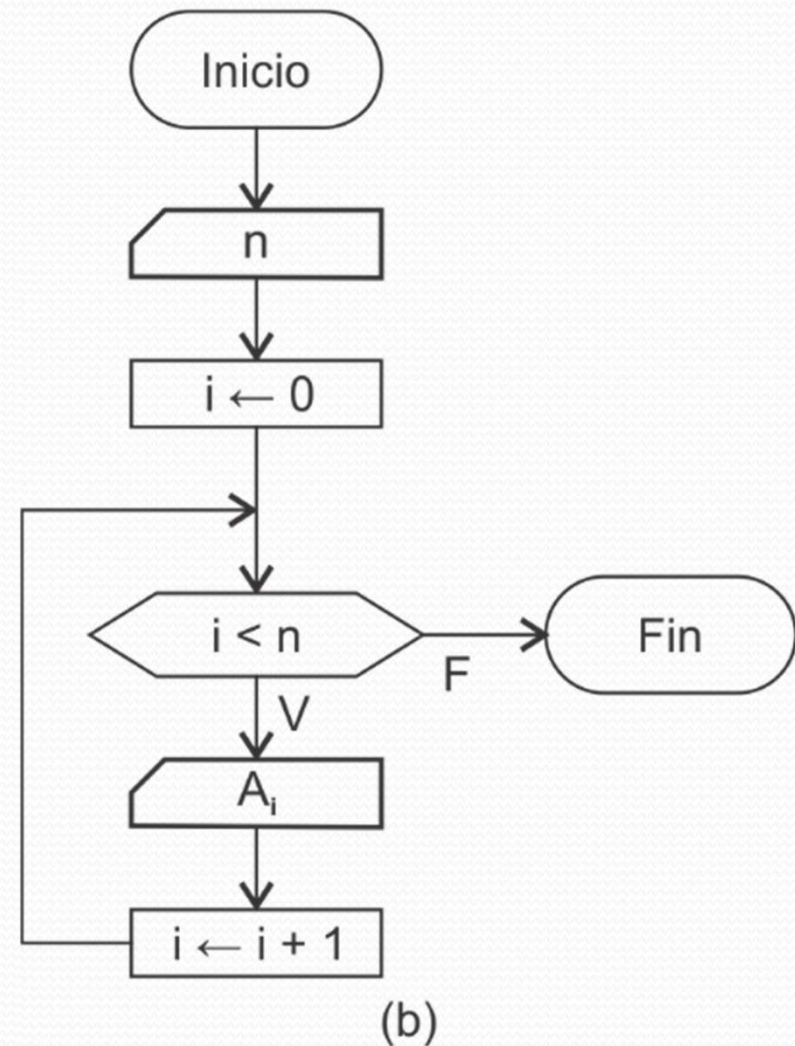
```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int n;          //Tamaño del arreglo
    int i;          //índice del arreglo

    //Paso 1: Se define la variable A como arreglo
    int A[10];
    //Paso 2: Se pide el número de datos y se almacena en n
    printf("Tamaño del arreglo: "); scanf_s("%d", &n);
    //Paso 3: Inicia el índice i en 0
    i = 0;
    do
    {
        //Paso 4: Se pide el valor de Ai
        printf("A[%d]=", i); scanf_s("%d", &A[i]);
        //Paso 5: Se incrementa el valor de i en 1
        i++;
        //Paso 6: ¿Es i < n? Si: ir al paso 4. No: ir al paso 7.
    } while (i < n);
    //Paso 7: Fin
}
```

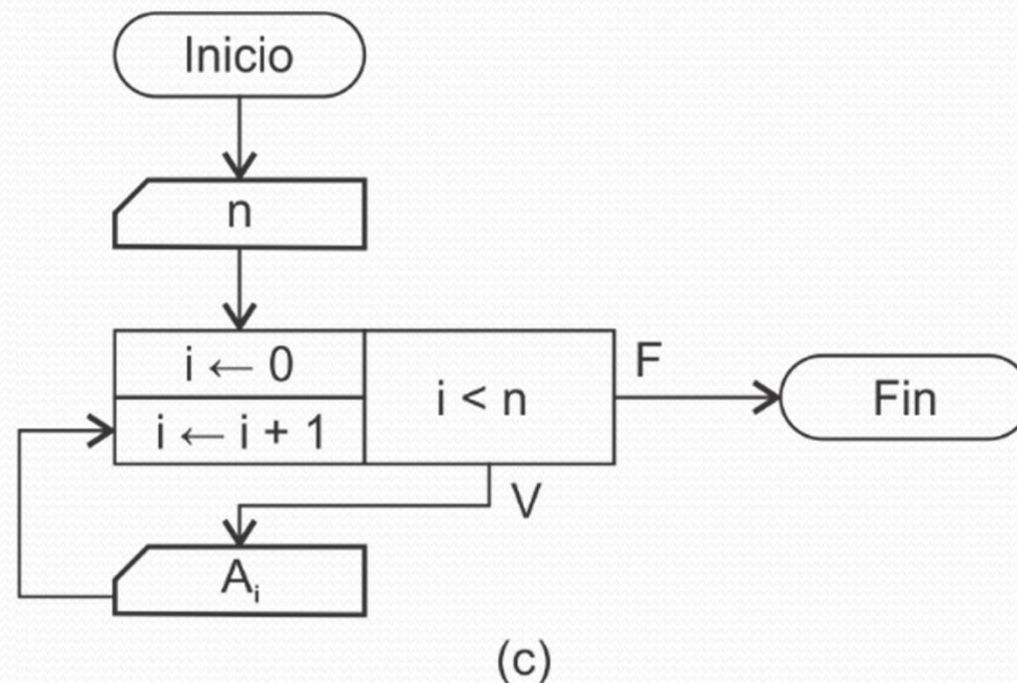

Arreglos unidimensionales.

- Asignación de datos a un vector. (a) Usando do - while. (b) Usando while. (c) Usando for.



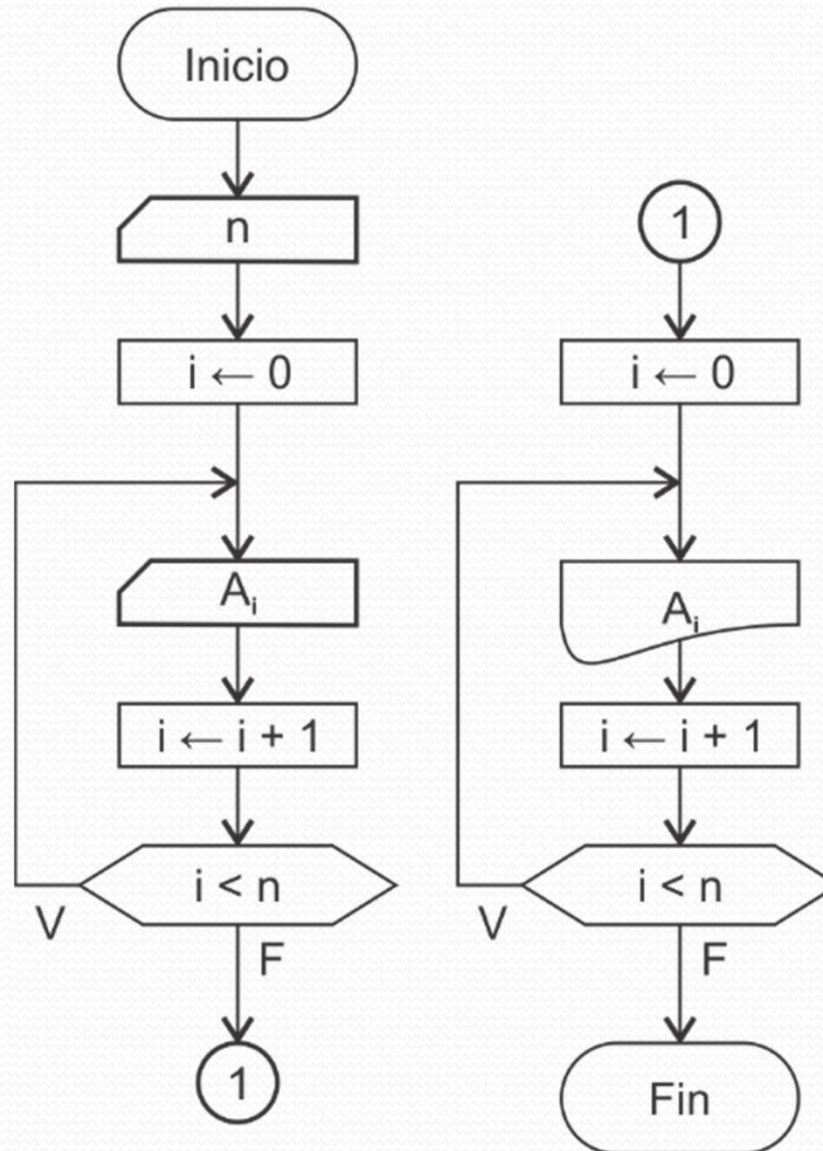
Arreglos unidimensionales.

- Asignación de datos a un vector. (a) Usando do - while. (b) Usando while. (c) Usando for.



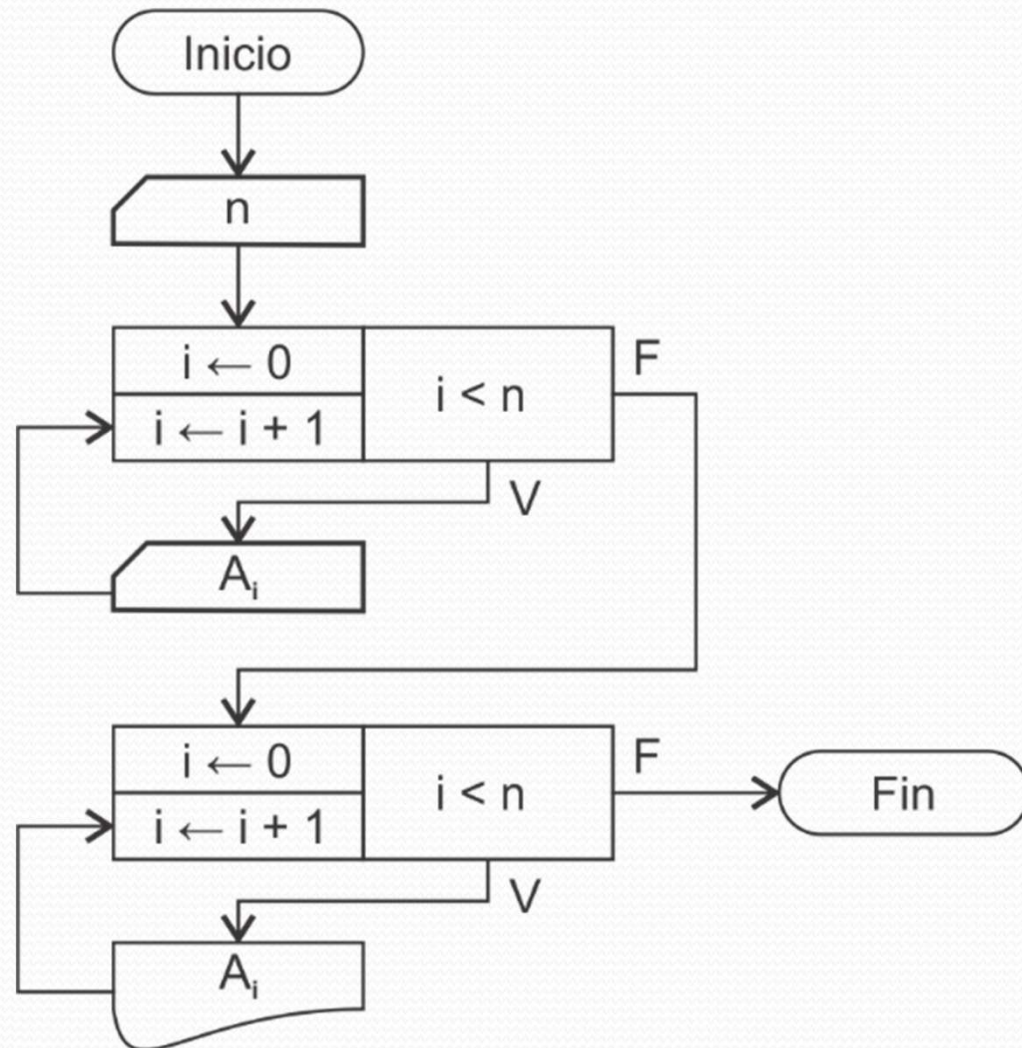
Asignación e impresión de datos en un vector

- Ciclo do – while



Asignación e impresión de datos en un vector

- Ciclo for



Ejemplo

```
void main()  
{  
    int a[10];  
    int i;  
  
    for (i = 0; i < 10; i++)  
        a[i] = 10 - i;  
    for (i = 0; i < 10; i++)  
        cout << "a[" << i << "] = " << a[i] << endl;  
}
```

Ejemplo. ¿Cuál es el problema?

```
void main()  
{  
    int ar[10];  
    int i;  
  
    for (i = 0; i < 11; i++)  
        ar[i] = 10 - i;  
    for (i = 0; i < 11; i++)  
        cout << "ar[" << i << "] = " << ar[i] << endl;  
}
```


Ejemplo. ¿Índice negativo?

```
void main()  
{  
    int ar[10];  
    int i;  
  
    for (i = -1; i < 10; i++)  
        ar[i] = 10 - i;  
    for (i = -1; i < 10; i++)  
        cout << "ar[" << i << "]= " << ar[i] << endl;  
}
```

Ejemplo. Predefinir valores a un vector.

```
void main()
{
    int ar[10] = { 1,2,3,4,5,6,7,8,9,10 };
    int i;

    for (i = 0; i < 10; i++)
        cout << "ar[" << i << "] = " << ar[i] << endl;
}
```


Ejemplo. Matriz.

```
void main()
{
    int ar[3][5], i, j, c=0;

    // Se asignan los valores a la matriz
    for (i = 0; i < 3; i++)
        for (j = 0; j < 5; j++){
            ar[i][j] = c; c++; }

    //Se imprimen los valores de la matriz
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 5; j++)
            cout << ar[i][j] << "\t";
        cout << "\n";}
}
```

Ejemplo. Matriz con valores iniciales.

```
void main()
{
    int ar[3][5] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14 };
    int i, j, c = 0;

    //Se imprimen los valores de la matriz
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 5; j++)
            cout << ar[i][j] << "\t";
        cout << "\n";
    }
}
```


Ejemplo. Direcciones de la matriz

```
void main()
{
    int ar[3][5] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14 };
    int i, j, c = 0;

    //Se imprimen las direcciones de la matriz
    for (i = 0; i < 3; i++){
        for (j = 0; j < 5; j++){
            cout << &ar[i][j] << endl;
            cout << "\n";
        }
    }
}
```


1.1.3 Arreglo de caracteres

- Un arreglo de caracteres también se conocen como “CADENA DE CARACTERES”.
- Un caracter es cualquier símbolo que maneja un código ASCII.
- Puede o no ser exhibido en un monitor.
- Se puede utilizar para almacenar información como nombres, o información en general.

```
S="ERA que se era";
```


1.1.3 Arreglo de caracteres

- Puede contener caracteres como:
 - Letras del alfabeto. Mayúsculas y/o minúsculas.
 - Números.
 - Caracteres aritméticos, lógicos, especiales, entre otros.
- El carácter nulo se representa como
 - NULL
 - '\0'
 - 0

1.1.3 Arreglo de caracteres

- Puede contener caracteres como:
 - Letras del alfabeto. Mayúsculas y/o minúsculas.
 - Números.
 - Caracteres aritméticos, lógicos, especiales, entre otros.
- El carácter nulo se representa como
 - NULL
 - '\0'
 - 0

Ejemplo. ¿Cuál es la salida?

```
#include <string.h>
```

```
void main()
```

```
{
```

```
    char s[22] = { "Programacion Avanzada" };
```

```
    int i;
```

```
    for (i = 0; i < strlen(s); i++)
```

```
        cout << s[i] << " ";
```

```
}
```

Ejemplo. Truncando la cadena.

```
void main()
{
    char s[22] = { "Programacion Avanzada" };
    int i;

    s[15] = '\\0';
    cout << s << " ";
}
```


Librería string.h

- Contiene funciones para trabajar con cadenas, por ejemplo:
 - Copiar
 - Buscar cadenas dentro de subcadenas
 - Comparar cadenas
 - Extraer subcadenas de una cadena
 - Medir la longitud de una cadena.

1.10 Apuntadores

- Una característica muy importante en C/C++, es el uso de apuntadores.
- Los apuntadores son variables que “apuntan” a la dirección de memoria de otras variables, y se pueden realizar operaciones directamente en memoria.
- Esto se traduce en rapidez de ejecución y mayor eficiencia.
- Para tratar este tema, es necesario revisar algunos términos técnicos.

1.10.1 *Tamaño de variables.*

- Antes de iniciar en el tema de apuntadores, debemos considerar que cada una de las variables tiene un determinado número de bytes, dependiendo del tipo de variable de que se trate.
- El Código 1.17 muestra la manera de obtener el tamaño en bytes de algunas variables en C/C++.

Código 1.17

```
#include <iostream>
using namespace std;
void main()
{
    int i,n;  char o;  float j;  double k;
    float a[5] = { 0.0, 0.0, 0.0, 0.0, 0.0 };
    i = j = k = 0;
    cout << "Tamaño en bytes de: " << endl;
    n = sizeof(i); cout << "int i = " << n << endl;
    n = sizeof(o); cout << "char o = " << n << endl;
    n = sizeof(j); cout << "float j = " << n << endl;
    n = sizeof(k); cout << "double k = " << n << endl;
    n = sizeof(a); cout << "Arreglo float a de 5 = " << n << endl;
}
```


Resultado del Código 1.17

Tamaño en bytes de:

`int i = 4`

`char o = 1`

`float j = 4`

`double k = 8`

`Arreglo float a de 5 = 20`

1.10.2 Dirección de memoria.

- Es el lugar físico en memoria asignado a todas y cada una de las variables que se está usando en el programa.
- Cuando se crea o declara una variable, el administrador de memoria le asigna espacio de memoria de acuerdo con la disponibilidad.
- Esta dirección de memoria es asignada por el administrador de memoria.

1.10.2 Dirección de memoria.

- Observe el Código 1.18.
- Se declaran diversos tipos de variables (int, float, char, double).
- Se obtiene la dirección de memoria correspondiente a cada uno.

1.10.2 Dirección de memoria.

- Observe el Código 1.18.
- Se declaran diversos tipos de variables (int, float, char, double).
- Se obtiene la dirección de memoria correspondiente a cada uno.

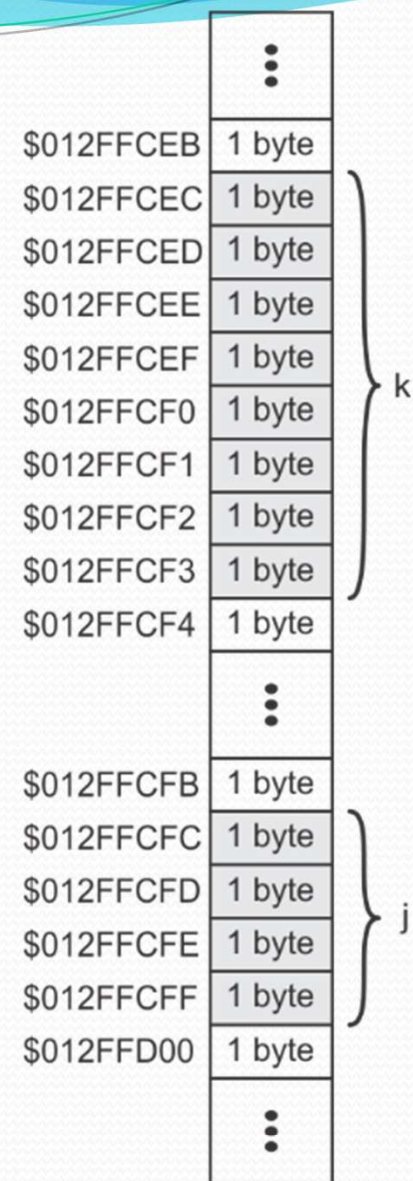
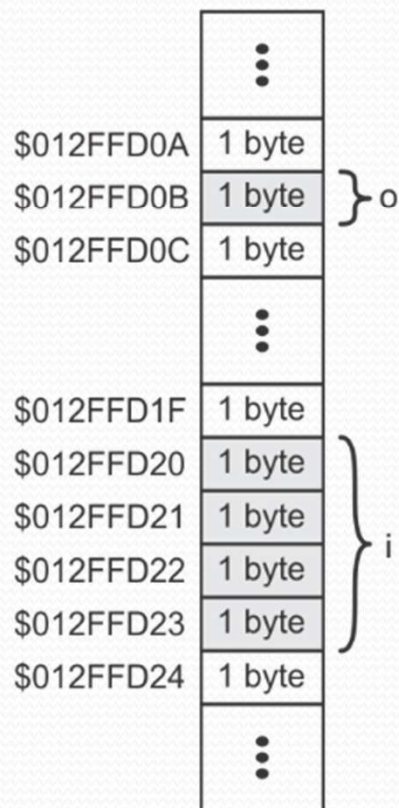
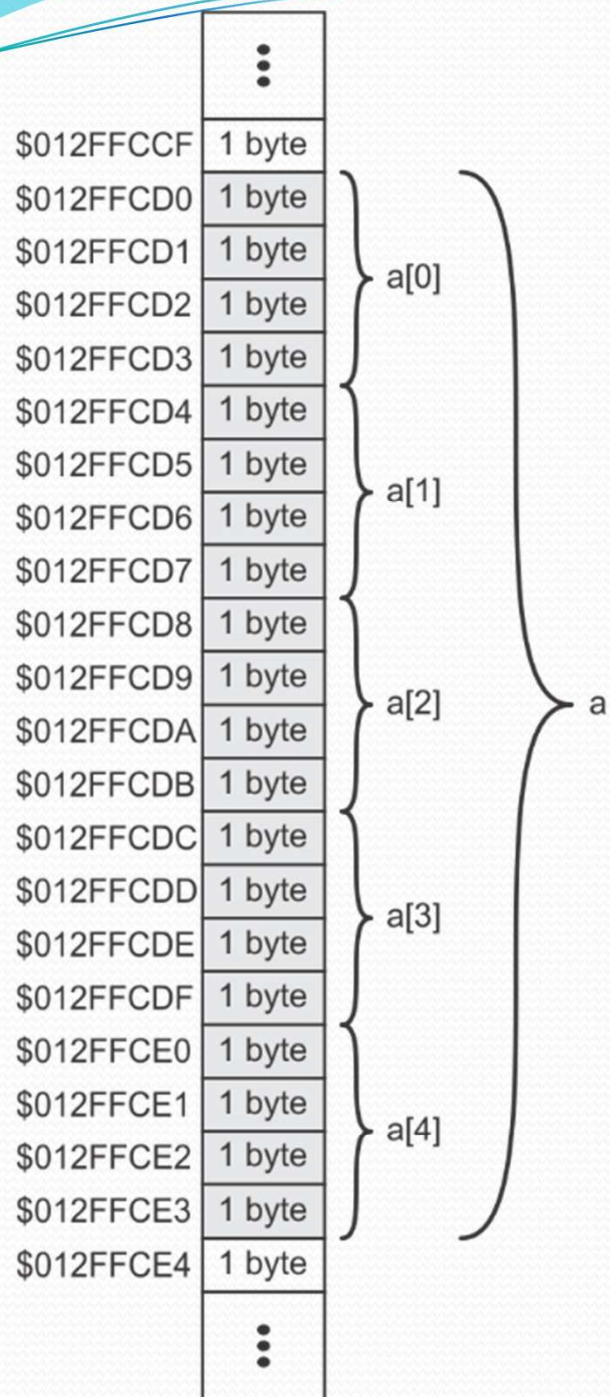
Código 1.18. Direcciones de memoria.

```
#include <iostream>
using namespace std;
void main()
{
    int i=0, n;      char o=1;      float j=0.0;
    double k=0;
    float a[5] = { 0.0, 0.0, 0.0, 0.0, 0.0 };

    cout << "Dirección de i = $" << &i << endl;
    printf("Dirección de o = $%p\n", &o);
    cout << "Dirección de j = $" << &j << endl;
    cout << "Dirección de k = $" << &k << endl;
    cout << "Dirección de a = $" << a << endl;
}
```


Código 1.18. Resultado

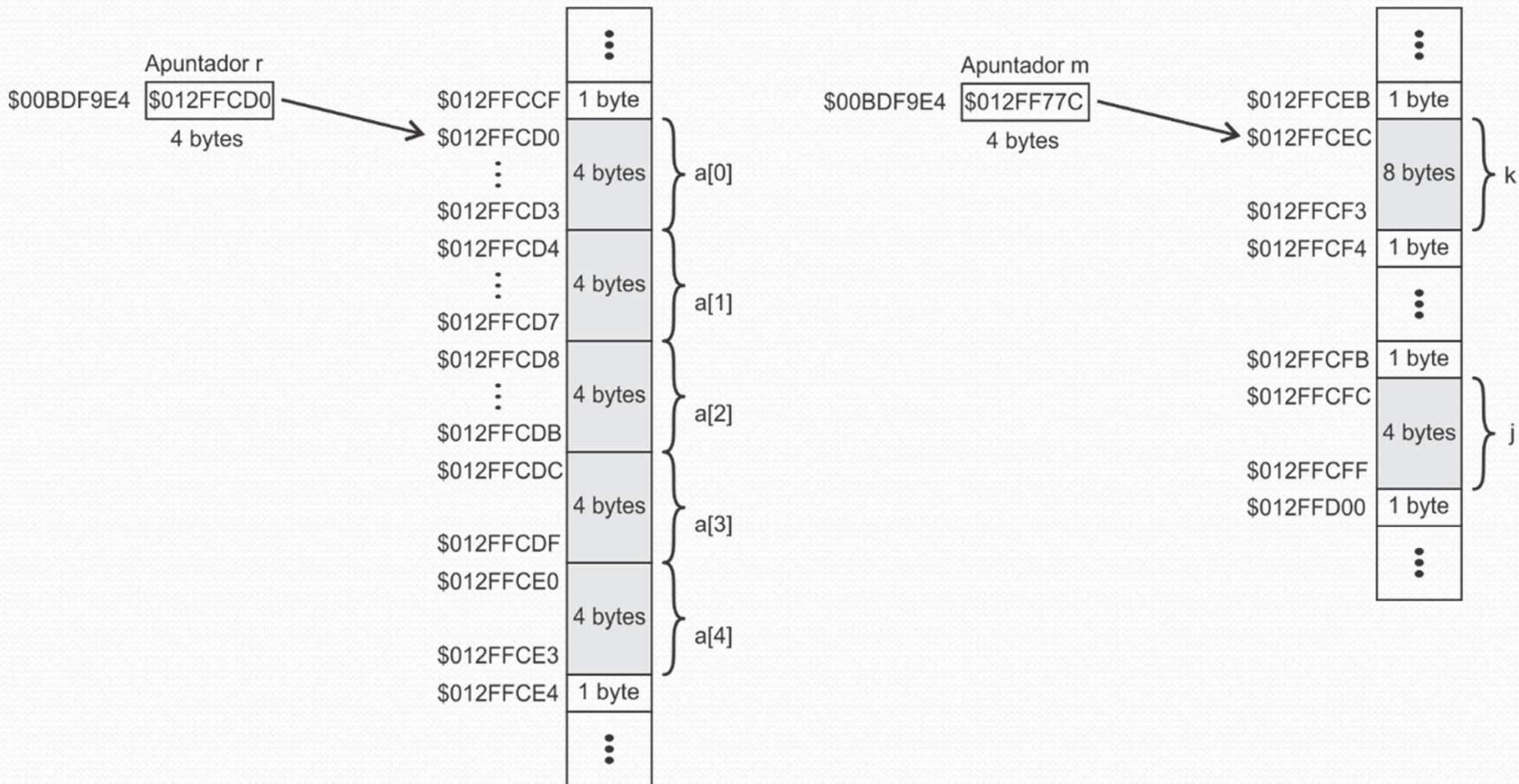
- Dirección de i = \$012FFD20
- Dirección de o = \$012FFD0B
- Dirección de j = \$012FFCFC
- Dirección de k = \$012FFCEC
- Dirección de a = \$012FFCD0



1.10.3 Apuntadores.

- Los apuntadores en C y C++ permite que los programas sean más eficientes debido a que se puede realizar operaciones directamente en memoria.
- Los apuntadores siempre deben “apuntar” a una dirección de memoria de una variable creada en el programa, ya sea variable local o global.

1.10.3 Apuntadores



1.10.3 Apuntadores

- La declaración de los apuntadores, se realiza de acuerdo a la siguiente sintaxis.

<tipo de dato> * <variable>

- El tipo de dato puede ser int, float, char, double, entre otros.
- El * indica que se está declarando un apuntador, y variable es propiamente el nombre del apuntador.

1.10.3 Apuntadores

- Observe los siguientes ejemplos.

`int *p; // Apuntador a un dato de tipo entero (int)`

`char *o, *u; // Apuntadores a datos de tipo
caracter (char)`

`float *f; // Apuntador a un dato de tipo punto-
flotante (float)`

1.10.3 Apuntadores

- Los punteros deben ser inicializados al asignar la dirección de memoria de la variable a la que se desea “apuntar”.
- Esto se realiza utilizando el operador de referenciación ‘&’, o al asignar direcciones de memoria almacenadas en otros apuntadores.

1.10.3 Apuntadores

- Ejemplo:

```
int i = 16;
```

```
int *p, *q;
```

```
// Se le asigna al puntero p la dirección de i
```

```
p = &i;
```

```
// Se asigna a q la dirección almacenada en p
```

```
// que es la dirección de i
```

```
q = p;
```


1.10.3 Apuntadores

- El acceso a la información a la que “apunta” un apuntador, se realiza anteponiendo a la variable el *, como se observa en el siguiente ejemplo.

```
int i = 231;
```

```
int j, *p;
```

```
p = &i;
```

```
// Imprime el valor de i
```

```
cout << "i = " << *p << endl;
```

```
j = *p - 100; // Se asigna el valor de i - 100
```

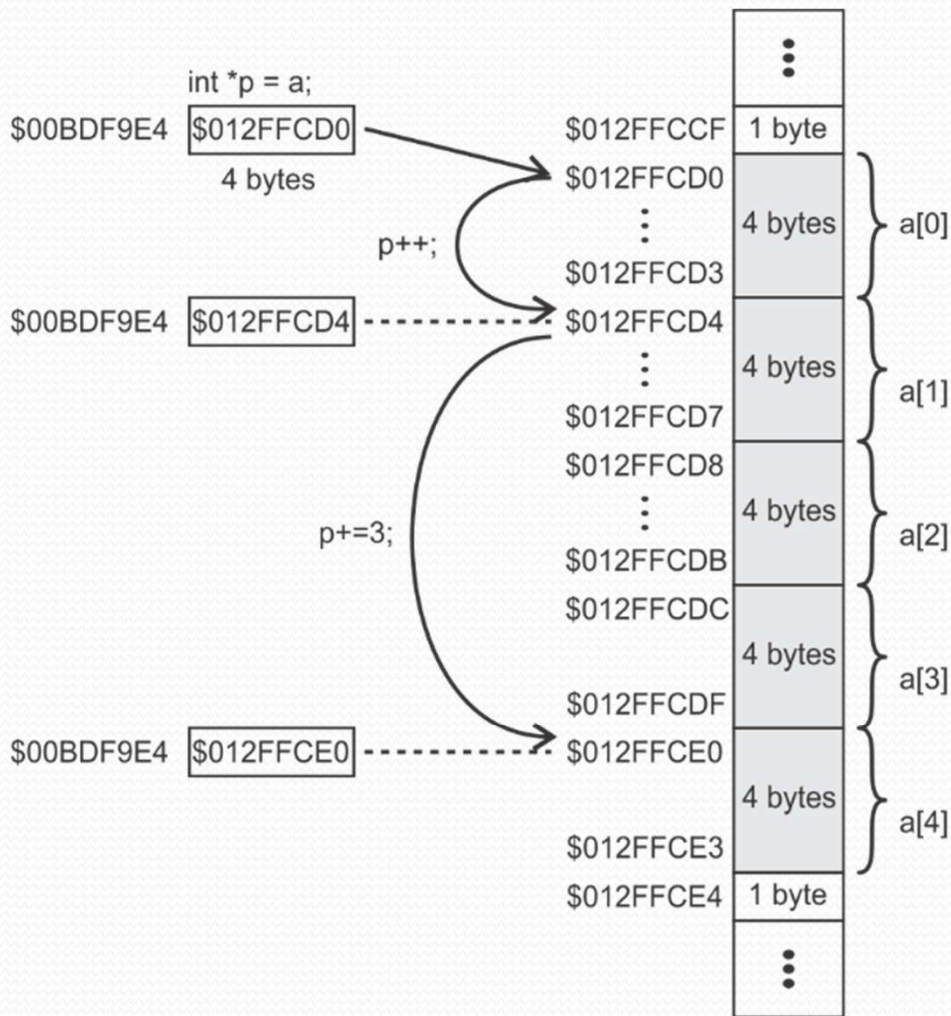

1.10.3 Apuntadores

- Los apuntadores también son muy utilizados en estructuras o registros. Pero se analizará su uso cuando se vea el tema de estructuras.

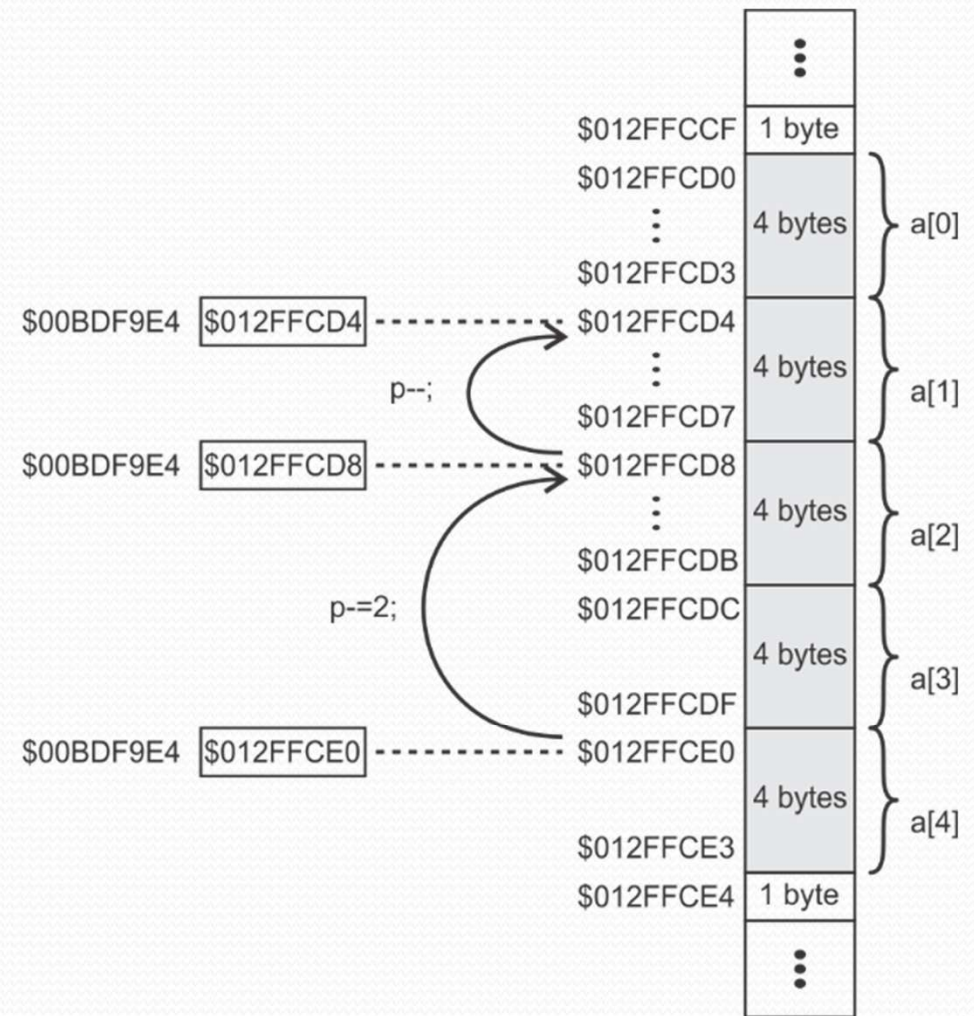
1.10.4 Operaciones con punteros.

- Las operaciones más comunes con punteros, son de suma y resta.
- Solamente se aplican estas operaciones en arreglos de datos.
- Se usa para mover al puntero a lo largo de un arreglo.

1.10.4 Operaciones con punteros.



(a)



(b)

1.10.4 Operaciones con punteros.

- Se debe tener cuidado al realizar operaciones de incremento / decremento en apuntadores, pues se puede salir del rango de memoria asignado, provocando errores en tiempo de ejecución.

Código 1.19. Asignación de valores a un arreglo.

```
void main()
{
    int *p; // Se declara el puntero
    // Se declara un vector a, del mismo tipo que el puntero
    int a[10];
    int i;

    p = a;
    for (i = 0; i < 10; i++)
        *p++ = i;
    for (i = 0; i < 10; i++)
        cout << a[i] << endl;
}
```

Código 1.20. Uso de punteros para imprimir un arreglo.

```
void main()
{
    int *p; // Se declara el puntero
    // Se declara un vector a, del mismo tipo que el puntero
    int a[10];
    int i;

    p = a;
    for (i = 0; i < 10; i++)
        *p++ = i;
    for (i = 0; i < 10; i++)
        cout << *p-- << endl;
}
```


Ejercicio. ¿Qué hace el programa?

```
void main()
{
    int *p; // Se declara el puntero
    int i; // Se declara una variable i, del mismo tipo que el puntero

    i = 5;
    cout << "Direccion de i = " << &i << endl;
    p = &i;
    cout << "i = " << i << endl;
    cout << "Direccion de p = " << p << endl;
    *p = 10;
    cout << "i = " << i << endl;
}
```


Ejercicio. ¿Cuál es el error?

```
void main()
{
    int *p; // Se declara el puntero
    int a[10]; // Se declara una variable i, del mismo tipo que el puntero
    int i;

    cout << "Con enteros\n";
    for (i = 0; i < 10; i++)
        a[i] = 10 - i;
    p = a;
    for (i = 0; i < 10; i++){
        cout << p << " Contiene " << *p << endl;
        p++;
    }
}
```


Ejercicio con flotante

```
void main()
{
    float *q;
    float b[10];
    int i;

    cout << "Con flotantes\n";
    for (i = 0; i < 10; i++)
        b[i] = i/10;
    q = b;
    for (i = 0; i < 10; i++){
        cout << q << " Contiene " << *q << endl; q++;
    }
}
```

Ejercicio con Double

```
void main()
{
    double *r, c[10];
    int i;

    cout << "Con dobles\n";
    for (i = 0; i < 10; i++)
        c[i] = i / 10;
    r = c;
    for (i = 0; i < 10; i++){
        cout << r << " Contiene " << *r << endl; r++;
    }
}
```


Ejercicio. ¿Cuál es la salida?

```
void main()
{
    int *p; // Se declara el puntero
    int a[10], i;

    p = a;
    for (i = 0; i < 10; i++) *p++ = i;
    for (i = 0; i < 10; i++)
        cout << a[i] << endl;
}
```

Ejercicio. Explique el programa.

```
void main()
{
    int *p; // Se declara el puntero
    int a[10], i;

    p = a;
    for (i = 0; i < 10; i++) *p++ = i;
    for (i = 0; i < 10; i++)
        cout << *p-- << endl;
}
```


Ejercicio. Corrija el código.

```
void main()
{
    int *p; // Se declara el puntero
    int a[10], i;

    p = a;
    for (i = 0; i < 10; i++) *++p = i;
    for (i = 0; i < 10; i++)
        cout << *p-- << endl;
}
```

Ejercicio. Punteros y funciones

```
void cambiar_dato(int *a, double *b){  
    *a = 10; *b = 35.675;  
}
```

```
void main(){  
    int *p,m;  
    double *q,n;  
  
    m = 12; n = 0.5;  
    cout << "m = " << m << "\t" << " n = " << n << endl;  
    p = &m; q = &n;  
    cambiar_dato(p, q);  
    cout << "m = " << m << "\t" << " n = " << n << endl;  
}
```


Ejercicio. Apuntadores y cadenas.

```
void main()
{
    char binario[10], *o;
    int i,n,m;

    cout << "Decimal a convertir a binario: ";
    n = 100; // Valor a convertir
    m = 11; // Numero de bits

    cout << "El valor binario de " << n << " es ";
    o = binario+m-2;
    *o-- = '\\0';
```

```
    for (i = 1; i < m-2; i++)
    {
        *o = (n % 2) + 48;
        o--;
        n /= 2;
    }
    *o = (n % 2) + 48;
    cout << binario << endl;
}
```


1.2 Memoria Dinámica

- C ofrece al desarrollador la opción de crear diferentes tipos de variables de forma dinámica.
- Se utilizan funciones como:
 - `malloc()`. Asigna un número determinado de bytes (bloque de memoria) a un apuntador.
 - `realloc()`. Modifica el tamaño del bloque de memoria reservado.
 - `calloc()`. Asigna un número determinado de bytes y se inicializa a cero.
 - `free()`. Libera el bloque de memoria, y el puntero queda como NULL.

Asignación de Memoria

- `char *caracter;`
 - `int *entero;`
 - `float *flotante;`
 - `double *doble;`
-
- `caracter = (char *)malloc(10*sizeof(char));`
 - `entero = (int *)malloc(10*sizeof(int));`
 - `flotante = (float *)malloc(10* sizeof(float));`
 - `doble = (double *)malloc(10* sizeof(double));`

Liberar Memoria

```
free(caracter);  
free(entero);  
free(flotante);  
free(doble);
```


Ejemplo. ¿Cuál es la salida?

```
void main(void)
{
    char *c;
    int i;

    //Uso de malloc para generar variables sencillas
    cout << "Reservando 10 localidades de memoria\n";
    c = (char *)malloc(10*sizeof(char));
    for (i = 0; i < 10; i++)
        c[i] = i+48;
    cout << c;
    free(c);
}
```

Ejemplo. ¿Cuántos byte se reservaron?

```
void main(void) {  
    int *A;  
    int i;  
  
    cout << "Reservando memoria\n";  
    A = (int *)malloc(10*sizeof(int));  
  
    for (i = 0; i < 10; i++)  
        A[i] = i;  
    cout << A;  
    free(A);  
}
```


Ejemplo. ¿Cuál es la salida?

```
void main(void) {  
    int *A;  
    int i;  
  
    cout << "Reservando memoria\n";  
    A = (int *)malloc(10*sizeof(int));  
  
    for (i = 0; i < 10; i++)  
        A[i] = i;  
    cout << A;  
    free(A);  
}
```

Ejemplo.

```
void main(void) {  
    int *A, i;  
  
    A = (int *)malloc(10*sizeof(int));  
    for (i = 0; i < 10; i++) A[i] = i;  
    for(i=0;i<10;i++)  
        cout << A[i] << endl;  
    free(A);  
}
```


Ejemplo. Aumentando la memoria.

```
void main(void) {  
    char *c;  
    int i;  
  
    c = (char *)malloc(10*sizeof(char));  
    for (i = 0; i < 10; i++) c[i] = i+48;  
    cout << c;  
    c = (char *)realloc(c, 15 * sizeof(char));  
    for (i = 10; i < 15; i++) c[i] = i + 48;  
    cout << c;  
    free(c);  
}
```

Ejemplo. Reduciendo memoria.

```
void main(void) {  
    char *c;  
    int i;  
  
    c = (char *)malloc(10 * sizeof(char));  
    for (i = 0; i < 10; i++) c[i] = i + 48;  
    cout << c;  
    c = (char *)realloc(c, 5 * sizeof(char));  
    cout << c;  
    free(c);  
}
```


1.3 Manejo de excepciones

- Las excepciones son errores en tiempo de ejecución.
- Si se genera un error en tiempo de ejecución y no se ha considerado la captura de errores, entonces:
 - El programa puede terminar abruptamente.
 - Si hay ficheros abiertos no se guarda el contenido de los buffers, ni se cierra el archivo.
 - Ciertos objetos no son destruidos, y no se libera memoria.
- Las excepciones más habituales son petición de memoria fallidas y operaciones matemáticas.

Sintaxis

```
try
{
    //Bloque de instrucciones 1
    throw tipo de error;
    //Bloque de instrucciones 2
}
catch (tipo variable)
{
    //Bloque de instrucciones 3
}
```


Ejemplo sencillo de manejo de excepciones

```
#include <iostream>
using namespace std;
```

```
void main()
{
    int i;
    float j;

    i = -2;
    j = (float) sqrt(i);
    cout << j;
}
```

Ejemplo sencillo de manejo de excepciones

```
void main()
{
    int i;
    float j;

    i = -2;
    j = (float) sqrt(i);
    cout << j;
}
```


Excepción desde una función fuera del bloque try.

```
void main()
{
    int i; float j;
    i = -2;      j = 0;
    try{
        if (i < 0) throw i;
        j = (float)sqrt(i);
        cout << j;}
    catch (int e) {
        cout << "No se puede obtener la raíz
cuadrada de un valor negativo " << e << endl;}}
```


try/catch pueden estar en una función

```
void Xhandler(int test){
    try{
        if(test) throw test;
    }
    catch(int i) {
        cout << "Capturada exepción nº: " << i << '\n';
    }
}

int main(){
    Xhandler(1);
    Xhandler(2);
    Xhandler(0);
    Xhandler(3);
    return 0;
}
```


Captura de distintos tipos de excepciones.

```
void Xhandler(int test)
{
    try{
        if(test) throw test;
        else throw "El valor es cero";
    }
    catch(int i) {
        cout << "Capturada excepción nº: " << i
        << '\n';
    }
    catch(const char *str) {
        cout << "Capturada una string: ";
        cout << str << '\n';
    }
}
```

```
int main()
{
    cout << "Inicio\n";
    Xhandler(1);
    Xhandler(2);
    Xhandler(0);
    Xhandler(3);
    cout << "Fin";
    return 0;
}
```


Usando catch(...) como defecto

```
void Xhandler(int test)
{
    try{
        if(test==0) throw test; // throw int
        if(test==1) throw 'a'; // throw char
        if(test==2) throw 123.23; // throw
double
    }
    catch(int i) { // catch an int exception
        cout << "Capturado un entero\n";
    }
    catch(...) { // catch all other
exceptions
        cout << "¡Una Capturada!\n";
    }
}
```

```
int main()
{
    cout << "Inicio\n";
    Xhandler(0);
    Xhandler(1);
    Xhandler(2);
    cout << "Fin";
    return 0;
}
```


Restringiendo tipos notificables

```
void Xhandler(int test)
throw(int, char, double)
{
    if(test==0) throw test; // int
    if(test==1) throw 'a'; // char
    if(test==2) throw 123.23;
    //double
}
```

```
int main()
{
    cout << "Inicio\n";
    try{
        Xhandler(0); // also, try
        passing 1 and 2 to Xhandler()
```

```
    }
    catch(int i) {
        cout << "Capturado
integer\n";
    }catch(char c) {
        cout << "Capturado
char\n";
    }
    catch(double d) {
        cout << "Capturado
double\n";
    }
    cout << "Fin";
    return 0;
}
```


Ejemplo completo

```
#include <iostream>
```

```
void divide(double a, double b);
```

```
int main()
```

```
{
```

```
    double i, j;
```

```
    do {
```

```
        cout << "Numerador (0 to stop): ";
```

```
        cin >> i;
```

```
        cout << "Denominador: ";
```

```
        cin >> j;
```

```
        divide(i, j);
```

```
    } while(i != 0);
```

```
    return 0;
```

```
}
```

```
void divide(double a, double b)
```

```
{
```

```
    try {
```

```
        if(!b) throw b; // check for divide-by-zero
```

```
        cout << "Resultado: " << a/b << endl;
```

```
    }
```

```
    catch (double b) {
```

```
        cout << "No se puede dividir por  
cero.\n";
```

```
    }
```

```
}
```


1.4 Sobrecargas de funciones

- Sobrecargar una función significa incluir más de una definición de la función.
- Que el compilador frente a una llamada a una función elija entre las diferentes definiciones cual es la correcta.
- C y C++ tiene muchas funciones sobrecargadas.

Ejemplo 1

```
int cubo (int);
double cubo(double);

int main()
{
    int ix = 5;
    double dx = 1.5;
    cout << "El cubo de " << ix << " es
" << cubo(ix) << endl;
    //el compilador al ver un valor int
    elige la funcion
    cout << "El cubo de " << dx << " es
" << cubo (dx) << endl;
    //la llamada es igual con un valor
    double.
    return 0;
}
```

```
//Sobrecarga para valores int
int cubo (int y ) {
    return y*y*y;
}

//Sobrecarga para valores double
double cubo (double y)
{
    return y*y*y;
}
```


Ejemplo 2

```
int mayor(int a, int b);
char mayor(char a, char b);
double mayor(double a, double b);

int main() {
    cout << mayor('a', 'f') << endl;
    cout << mayor(15, 35) << endl;
    cout << mayor(10.254, 12.452)
    << endl;

    return 0;
}
```

```
int mayor(int a, int b) {
    if(a > b) return a;
    else return b;
}

char mayor(char a, char b) {
    if(a > b) return a;
    else return b;
}

double mayor(double a, double b)
{
    if(a > b) return a;
    else return b;
}
```


Ejemplo 3

```
int mayor(int a, int b);  
int mayor(int a, int b, int c);  
int mayor(int a, int b, int c, int  
d);
```

```
int main() {  
    cout << mayor(10, 4) <<  
endl;  
    cout << mayor(15, 35, 23)  
<< endl;  
    cout << mayor(10, 12, 12,  
18) << endl;  
  
    return 0;  
}
```

```
int mayor(int a, int b) {  
    if(a > b) return a;  
    else return b;  
}
```

```
int mayor(int a, int b, int c) {  
    return mayor(mayor(a, b), c);  
}
```

```
int mayor(int a, int b, int c, int d) {  
    return mayor(mayor(a, b),  
                mayor(c, d));  
}
```


Ejercicios.

- Realice un programa en C, que realice lo siguiente:
 1. Ordene una lista de n número de mayor a menor, utilizando punteros. El tamaño del vector debe ser variable.
 2. Convertir un número decimal a binario, considerando n bits de la parte entera y m bits de la parte fraccionaria. Use punteros.
 3. Convertir un valor doble o entero en binario. Use sobrecarga de funciones y memoria dinámica.

Manejo de constantes

- El lenguaje C permite definir constantes.
- La sintaxis de declaración es la siguiente.
 - `#define nombre valor`
- `#define` es la directiva usada.
- *Nombre* es la referencia de la constante. Se crea bajo las reglas de una variable, pero NO puede cambiar su valor.
- *Valor* es el dato que se asigna a la constante.

Ejemplo

```
#include <iostream>

#define pi 3.141516

using namespace std;

int main()
{
    float r, a;

    r = (float)2.0;
    a = (float)(pi * r * r);
    cout << a;
}
```

Macros

- El uso de macros en C / C++, es una característica muy importante.
- Permite definir funciones.
- Usa la directiva `#define`.

Ejemplo de Macros

```
#include <iostream>
#define pi 3.141516
#define f(x) x*x*pi

using namespace std;

void main()
{
    float r, a;

    r = (float)2.0;
    a = f(r);
    cout << a;
}
```


Ejemplo de Macros

```
<iostream>
#include <math.h>
#define f1(A,B,C)  sqrt(B*B-4*A*C) / (2*A)
#define f2(A,B)    -B / (2*A)
using namespace std;
void main()
{
    double a, b, c;    double x1, x2;
    a = 12.0; b = -36.0; c = -120.0;
    x1 = f2(a, b) + f1(a, b, c);
    x2 = f2(a, b) - f1(a, b, c);
    cout << x1 << endl << x2 << endl;
}
```


Ejemplo de Macros

```
#include <iostream>
#include <math.h>
#define f1(A,B,C)  sqrt(B*B-4*A*C) / (2*A)
#define f2(A,B)    -B / (2*A)
#define g1(A,B,C)  f2(A,B)+f1(A,B,C)
#define g2(A,B,C)  f2(A,B)-f1(A,B,C)
using namespace std;
void main() {
    double a, b, c;
    double x1, x2;
    a = 12.0; b = -36.0; c = -120.0;
    x1 = g1(a, b, c);
    x2 = g2(a, b, c);
    cout << x1 << endl << x2 << endl;
}
```